

Version Control (Git)

Professional Git Workflow for Real Projects

■ LEARNING OUTCOME

By the end of this module you will use Git professionally for web projects – branching, committing with meaningful messages, resolving merge conflicts, using GitHub for collaboration, and implementing a CI-friendly workflow.

■ Skills You Will Gain

- Initialise repos and stage/commit with meaningful messages
- Use branches for features, fixes, and releases
- Merge and rebase branches correctly
- Resolve merge conflicts confidently
- Use GitHub: pull requests, code review, issues
- Set up .gitignore and protect secrets

■ Git is the number one professional skill that beginners skip. Every job uses Git. Every team uses pull requests and code review. Messy commits and force-pushes to main tell recruiters you work alone.

SECTION 1

Git Essentials

```
# SETUP (once per machine) git config --global user.name 'Alice Sharma' git config --global user.email 'alice@example.com' git config --global init.defaultBranch main git config --global core.editor 'code --wait' # VS Code as editor # INITIALISE git init # new repo in current folder git clone https://github.com/user/repo.git # clone existing repo # DAILY WORKFLOW git status # what has changed? git diff # see exact changes not yet staged git add . # stage all changes git add src/components/Button.jsx # stage specific file git add -p # interactive stage (chunk by chunk) git commit -m 'feat: add search filter to product list' git push origin main # push to remote git pull origin main # fetch + merge from remote git fetch origin # fetch without merging # UNDO MISTAKES git restore filename.js # discard unstaged changes to file git restore --staged filename.js # unstage file (keep changes) git commit --amend --no-edit # add more changes to LAST commit git reset HEAD~1 # undo last commit, keep changes staged git reset --hard HEAD~1 # undo last commit, LOSE all changes git revert abc1234 # create new commit that undoes abc1234 # Rule: never force-push to shared branches!
```

SECTION 2

Branching Strategy

Git Flow -- Professional Branching Model

main: Production-ready code only. Protected. No direct pushes. develop: Integration branch. Features merge here first. feature/: Individual features. Branch from develop, merge to develop. fix/: Bug fixes. Branch from main (hotfix) or develop. release/: Pre-release testing. Branch from develop, merge to main + develop. Simplified Flow (for solo/small teams): main: Production. Protected. feature/branch-name: Every feature or fix gets its own branch. Rule: main is always deployable. Every change goes through a PR.

```
# BRANCHING git branch # list local branches git branch -a # list all branches (including remote) git checkout -b feature/user-auth # create AND switch to new branch git switch main # switch to main branch (modern syntax) git switch -c fix/broken-login # create and switch (modern) # MERGING git switch main git merge feature/user-auth # merge feature into current branch git merge --no-ff feature/user-auth # always create merge commit (recommended) git merge --squash feature/user-auth # squash all commits into one # REBASING (rewrite history for clean log) git switch feature/user-auth git rebase main # replay feature commits on top of main git rebase -i HEAD~3 # interactive rebase -- squash/edit last 3 commits # Rebase vs Merge: # Merge: preserves full history, adds merge commit # Rebase: linear history, no merge commit, rewrites commit hashes # Rule: rebase local feature branches; merge for shared branches # STASH -- temporarily save changes without committing git stash # stash current changes git stash push -m 'WIP: search feature' # with description git stash list # see all stashes git stash pop # apply and remove latest stash git stash apply stash@{2} # apply specific stash (keep it) git stash drop stash@{0} # delete specific stash # TAGS -- mark releases git tag v1.0.0 # lightweight tag git tag -a v1.0.0 -m 'Release 1.0.0 -- initial launch' # annotated git push origin v1.0.0 # push specific tag git push origin --tags # push all tags
```

SECTION 3

Merge Conflicts and Resolution

```
# MERGE CONFLICTS -- happen when both branches edit the same line git merge feature/update-navbar # AUTO-MERGE FAILED -- resolve conflicts, then commit # Conflict markers in file: <<<<<< HEAD (your branch -- current) const navItems = ['Home', 'About', 'Contact']; ===== const navItems = ['Home', 'Products', 'Blog', 'Contact']; >>>>>> feature/update-navbar (incoming branch) # RESOLVE: # 1. Edit the file to what you want it to be const navItems = ['Home', 'Products', 'About', 'Blog', 'Contact']; # 2. Remove ALL conflict markers (<<<, ==, >>>) # 3. git add the resolved file # 4.
```

```
git commit git add src/components/Navbar.jsx git commit -m 'merge: resolve navbar items conflict' #
ABORT a merge if it gets too complex git merge --abort # TOOLS for conflict resolution git
mergetool # open visual merge tool # VS Code: shows inline Accept Current / Accept Incoming /
Accept Both # Tip: keep commits small and focused -- fewer conflicts!
```

SECTION 4

GitHub Workflow: Issues, PRs, Code Review

```
# COMMIT MESSAGE CONVENTIONS (Conventional Commits) # Format: type(scope): description # Types:
feat, fix, docs, style, refactor, test, chore, ci, perf feat: add user authentication with JWT
feat(auth): add Google OAuth login option fix(cart): correct total price calculation with discount
docs: update README with deployment instructions refactor(api): extract user service from
controller test: add unit tests for email validation chore: upgrade React to v18.2 ci: add GitHub
Actions workflow for automated testing # GOOD commit messages: # - Imperative mood: 'add feature'
not 'added feature' # - Explain WHY in the body, not just WHAT # - 50 chars or less for subject line
# - Reference issue numbers: 'fix: correct login redirect (#42)' # PULL REQUEST TEMPLATE
(.github/PULL_REQUEST_TEMPLATE.md) ## What does this PR do? Brief description of the change. ##
Type of change - [ ] Bug fix - [ ] New feature - [ ] Breaking change - [ ] Documentation ## How to
test 1. Step one 2. Step two ## Checklist - [ ] Tests pass - [ ] Linter passes - [ ] README updated
if needed # GITHUB CLI -- manage GitHub from terminal npm install -g gh gh repo create my-project
--public gh issue create --title 'Add dark mode' --body 'Users have requested...' gh pr create
--title 'feat: add dark mode' --body 'Resolves #23' gh pr merge --squash # merge PR from terminal
```

SECTION 5

gitignore and Security

```
# .gitignore -- files Git should NEVER track # Node.js node_modules/ npm-debug.log* yarn-error.log
# Environment variables -- NEVER commit secrets! .env .env.local .env.*.local # Build outputs dist/
build/ .next/ # OS files .DS_Store Thumbs.db # Editor .vscode/settings.json *.swp # Logs logs/
*.log # Test coverage coverage/ # WHAT TO COMMIT INSTEAD # .env.example -- template with
placeholder values DATABASE_URL=postgresql://user:password@localhost:5432/mydb
JWT_SECRET=your-secret-key-here VITE_API_URL=http://localhost:3000 SMTP_HOST=smtp.gmail.com # If
you accidentally committed a secret: git filter-branch --force --index-filter \ 'git rm --cached
--ignore-unmatch .env' HEAD # Better: use BFG Repo Cleaner # ALWAYS rotate the secret after it was
exposed!
```

■ PRACTICE Q & A

Q1. What is the difference between git merge and git rebase?

A: merge: combines branches by creating a new merge commit -- preserves exact history. rebase: replays your commits on top of another branch -- creates linear history. Use merge for public/shared branches; rebase to clean up local feature branches before merging.

Q2. What is the difference between git reset and git revert?

A: reset: rewrites history by moving the branch pointer -- dangerous on shared branches. revert: creates a new commit that undoes changes -- safe for shared branches. Rule: never reset after pushing to a shared branch.

Q3. Why should you use feature branches instead of committing to main?

A: Feature branches isolate work in progress from stable code. They enable code review via pull requests, parallel development without conflicts, easy rollback of a feature, and CI/CD checks before code reaches production.

Q4. What should a good commit message contain?

A: Subject line (50 chars max): type(scope): imperative description. Body: why this change was needed, not what was changed (the diff shows that). Footer: reference to issue numbers. Never: 'fix stuff', 'WIP', 'asdfgh'.

Q5. What is a .gitignore and what should always be in it?

A: Tells Git which files not to track. Always include: node_modules/, .env files, build outputs (dist/, .next/), OS files (.DS_Store), editor settings. .env files especially -- committing secrets is a serious security incident.

■ Git Cheat Sheet

<code>git checkout -b feature/name</code>	Create + switch to feature branch
<code>git add -p</code>	Stage changes interactively
<code>feat: add search filter</code>	Conventional commit format
<code>git stash / git stash pop</code>	Save/restore WIP changes
<code>git merge --no-ff feature</code>	Merge with explicit merge commit
<code>git rebase -i HEAD~3</code>	Squash/edit last 3 commits
<code>git reset HEAD~1</code>	Undo last commit, keep changes
<code>git revert abc123</code>	Safe undo on shared branches
<code>git push origin --tags</code>	Push all tags to remote
<code>gh pr create</code>	Create GitHub PR from terminal
<code>.env</code> in <code>.gitignore</code>	NEVER commit secrets
<code>.env.example</code>	Commit this template instead